Analiza eficienţei algoritmilor: exerciții

Universitatea “Alexandru Ioan Cuza”, Iași

February 24, 2026

Amintiți-vă că pentru demonstrarea corectitudinii unui algoritm, este nevoie să analizați invariantul/invarianții (cu proprietățile de inițializare, menținere, terminare).

Asigurați-vă că înțelegeți bine problema de rezolvat înainte de a scrie un algoritm sau a analiza un algoritm propus pentru ea. Vă poate ajuta să înțelegeți cum arată posibile inputuri pentru problemă.

1. Fie următoarea problemă: dându-se un vector de numere naturale cu cel puțin două elemente, să se afișeze doi indici, pentru cele mai mici două elemente din vector, în ordine după valoarea elementelor. Exemplu: Pentru [3, 5, 2, 7, 4, 1] se va afișa 5, 2.

   (a) Foormalizați problema ca pereche Input-Output.

   (b) Este corect următorul algoritm pentru rezolvarea problemei? Justificați.

   def FirstSmallestInd(a):s

   ```
   n ← a.size()
   if a[0] < a[1] then
       sml ← 0
       nxtsml ← 1;
   else
       sml ← 1
       nxtsml ← 0
   for i ← 2..n − 1 do
       if a[i] < a[sml] then
           nxtsml ← sml
           sml ← i
   print sml
   print nxtsml
   ```

2. Un punct fix într-un vector este un indice i astfel încât arr[i] == i. Dându-se un vector sortat de numere întregi distincte, proiectați un algoritm pentru a găsi un punct fix dacă există unul sau returnați -1 în caz contrar. Poți mai bine decât O(n)?

3. Un element majoritar dintr-un vector de dimensiune n este un element care apare de mai mult de n/2 ori. Dându-se un vector cu n elemente

naturale, returnaţi elementul majoritar dacă există, sau -1 dacă nu există. Fie următorul algoritm pentru problema dată:

```
def getMajEl(A):
candidate ← −1
count ← 0
n ← A.size
for i ← 0..n − 1 do
    if count = 0 then
        candidate ← A[i]
        count ← 1
    else
        if candidate = A[i] then
            count ← count + 1
        else
            count ← count − 1
occurrences ← 0
for i ← 0..n − 1 do
    if A[i] = candidate then
        occurrences ← occurrences + 1
if occurrences > ⌊n/2⌋ then
    return candidate
else
    return −1
```

(a) Este algoritmul corect?

(b) De ce se folosesc două parcurgeri ale vectorului?

(c) Care este complexitatea timp a algoritmului?

4. Operații pe numere mari.

   (a) Enunțați problemele adunării/înmulțirii a două numere naturale, numere reprezentate printr-un șir de cifre.

   (b) Scrieți algoritmi pentru problemele de mai sus.

   (c) Calculați timpul de calcul în cazul ce mai nefavorabil pentru cei doi algoritmi de mai sus, în funcție de dimensiunea datelor de intrare.

   (d) Problemă mai grea: scrieți algoritmul lui Karatsuba pentru înmulțirea a două numere naturale, de complexitate $O(n^{log_2(3)})$.

5. Enunțați și formalizați problema sortării în ordine crescătoare a unui șir de numere naturale.

   (a) Scrieți algoritmii bubble-sort și selection sort pentru problema de mai sus.

   (b) Arătați că algoritmii de mai sus rezolva într-adevăr problema.

(c) Calculați timpul de execuție pentru algoritmii de sortare de mai sus pentru intrarea dată de vectorul $1, 2, \ldots, n$ și pentru $n, n-1, \ldots, 1$, unde n este un parametru în funcție de care trebuie calculat timpul de calcul.

6. Găsiți soluții cât mai eficiente din punct de vedere al timpului de calcul și al necesarului de memorie pentru următoare problemă: dându-se un număr natural n și $n-1$ numere naturale distincte între 1 și n să se determine numărul lipsă. Formalizați problema ca pereche `Input-Output`. Arătați că algoritmul (sau algoritmii) propuși rezolvă într-adevăr problema; calculați timpul de calcul și necesarul de memorie.

7. Același exercițiu ca mai sus, dar de data aceasta un număr apare de două ori și un număr lipsește. Formalizați problema.

8. Formalizați următoarele reduceri:

   (a) cmmmc la cmmdc,

   (b) egalitatea de mulțimi la sortare,

   (c) disjuncția de mulțimi la sortare.

9. Cum definim complexitatea $O(f(n))$ a unei probleme? Dar $\Omega(f(n))$?

10. Ce înseamnă algoritm optim pentru o problemă?

11. Arătați că dacă un algoritm A rezolvă o problemă P în timp $O(f)$ și $f = O(g)$, atunci problema P are complexitatea $O(g)$.

12. Justificaţi, plecând de la definiţie, că $0.5n^2 - 3n = \theta(n^2)$.

13. Arătați că dacă f este polinom de grad k, atunci $f(n) \in \theta(n^k)$. f

14. Arătați că $\theta(f) = O(f) \cap \Omega(f)$ pentru orice funcție f.

15. Fie $T(n) = \begin{cases} \theta(1) & \text{dacă } n = 1 \\ 2T(n/2) + \theta(n) & \text{dacă } n > 1. \end{cases}$

    Arătaţi că $T(n) = \theta(nlog(n))$.

16. Arătaţi prin inducţie că următoarea funcţie întoarce rezultatul $3^n - 2^n$:

```
f(n) {
  if (n == 0 || n == 1) {
    return n;
  }
  return 5 * f(n - 1) - 6 * f(n - 2);
}
```

17. Scrieți algoritmul căutării Fibonacci pentru problema căutării unei valori într-un vector sortat și testați-l pe câteva exemple. (google search). Analizați complexitatea algoritmului relativ la complexitatea căutării binare obișnuite. Sunt algoritmii la fel de eficienți din punct de vedere asimptotic în cazul cel mai nefavorabil? Ce concluzie se poate trage despre complexitatea problemei căutării într-un vector ordonat?

18. Implementaţi algoritmul heapsort și scrieți un număr de teste relevante care să arate că algoritmul funcționeaza conform descrierii problemei (cel puțin pentru cazurile de test).